Mastering the Vim Editor

A Comprehensive 3-Day Training Guide

From Absolute Beginner to Expert Mastery

```
╔═══════════════════════════════════════════════════════════╗
║                                                           ║
║    ▐▌   ▐▌▐▌▐▌▐▌   ▐▌▐▌    ▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌  ║
║    ▐▌   ▐▌▐▌▐▌▐▌▐▌▐▌▐▌    ▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌  ║
║    ▐▌   ▐▌▐▌▐▌▐▌▐▌▐▌▐▌    ▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌  ║
║     ▐▌ ▐▌ ▐▌▐▌▐▌▐▌ ▐▌▐▌    ▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌  ║
║      ▐▌▐▌  ▐▌▐▌▐▌  ▐▌▐▌    ▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌▐▌  ║
║                                                           ║
║          MASTERING THE VIM EDITOR                         ║
║          3-Day Intensive Training Program                 ║
╚═══════════════════════════════════════════════════════════╝
```

📋 Program Overview

Day	Focus	Topics
Day 1	Vim Philosophy, Modes & Core Navigation	Sections 1.0 – 1.3
Day 2	Editing, Searching, Macros & Configuration	Sections 2.0 – 2.3
Day 3	Advanced Features, Plugins & Real-World Mastery	Sections 3.0 – 3.3

Target Audience: Complete beginners with no Vim experience
Format: Hands-on, example-driven, visually rich
Goal: Go from "I can't even exit Vim" to "Vim is my superpower"

📚 Table of Contents

- Section 1.0 – Introduction to Vim
- Section 1.1 – Vim's Modal Architecture
- Section 1.2 – Navigation: Moving at the Speed of Thought
- Section 1.3 – Inserting, Deleting & Basic Editing
- Section 2.0 – The Vim Language: Operators, Motions & Text Objects
- Section 2.1 – Searching, Finding & Replacing
- Section 2.2 – Registers, Macros & Automation
- Section 2.3 – Vim Configuration & the vimrc
- Section 3.0 – Buffers, Windows, Tabs & the File System
- Section 3.1 – Advanced Editing Techniques
- Section 3.2 – Plugins & the Modern Vim Ecosystem

- Section 3.3 – Real-World Vim Workflows
- Complete Vim Cheat Sheet

Section 1.0

Introduction to Vim

🎯 Learning Objectives

- Understand what Vim is and why it's still widely used after 30+ years
- Know the difference between Vim, Vi, and Neovim
- Appreciate the philosophy behind modal editing
- Successfully open, navigate, and exit Vim for the first time

---

What is Vim?

Vim (**Vi Im**proved) is a highly configurable, keyboard-driven text editor that has been a cornerstone of programming and system administration since 1991. Its predecessor, **vi**, dates back to 1976. Despite being nearly 50 years old in concept, Vim (and its modern successor **Neovim**) remains one of the most-used editors among professional developers, system administrators, and power users worldwide.

```
THE VIM FAMILY TREE
═══════════════════════════════════════════════════════════

1969: ed        → Line editor on UNIX (the original)
      │
1976: vi        → Visual editor (Bill Joy, Berkeley)
      │           The first screen-based editor
      │
1991: Vim       → Vi Improved (Bram Moolenaar)
      │           Added syntax highlighting, plugins, much more
      │
2015: Neovim    → Modern Vim fork
                  Better plugin system, LSP built-in, Lua config
                  (Drop-in replacement — all Vim knowledge transfers!)
```

Why Learn Vim in 2024?

You might wonder: with modern editors like VS Code and JetBrains IDEs, why bother with Vim? Here's why:

```
REASONS TO LEARN VIM
═══════════════════════════════════════════════════════════════

1. SPEED — Expert Vim users edit text faster than any other editor.
   The keyboard-centric design means your hands never leave the keys.
   No mouse. No menu hunting. Just pure, fluid editing.

2. UNIVERSALITY — Vim is on EVERY Linux/Mac server by default.
   When you SSH into a remote machine, Vim is there.
   You can't say that about VS Code.

3. COMPOSABILITY — Vim commands combine like words in a language.
   "Delete 3 words": d3w
   "Change inside quotes": ci"
   "Yank to end of line": y$
   Once you know the grammar, new commands feel obvious.

4. CUSTOMIZABILITY — Vim is infinitely configurable.
   Your editor grows with you. Plugins add any feature you want.

5. LONGEVITY — Skills you build in Vim last decades.
   You won't have to relearn when an IDE changes its interface.

6. IT'S EVERYWHERE — Vim keybindings exist in:
   • Bash / Zsh shells (set -o vi)
   • VS Code (Vim extension — most-installed extension!)
   • JetBrains IDEs (IdeaVim plugin)
   • Chrome/Firefox (Vimium extension)
   • Jupyter Notebooks, Gmail, and more
```

Vim vs. VS Code vs. Other Editors

Feature	Vim/Neovim	VS Code	Notepad++
Learning curve	Steep	Gentle	Gentle
Speed (expert)	Fastest	Fast	Medium
Available everywhere	✅ Always	❌ Must install	❌ Windows only
Customizability	Infinite	High	Medium
Resource usage	Minimal (5MB)	Heavy (500MB+)	Light
Remote editing	✅ Native (SSH)	⚠️ Extension needed	❌ No
Plugin ecosystem	Massive	Massive	Moderate
Initial productivity	Low	High	High
Long-term productivity	Highest	High	Medium

Installing Vim

```
# macOS (Homebrew — recommended for latest version)
brew install vim
# Or install Neovim (modern, recommended for new learners):
brew install neovim      # Command: nvim

# Ubuntu / Debian Linux
sudo apt update
sudo apt install vim

# CentOS / RHEL / Fedora
sudo dnf install vim

# Windows
# Option 1: Download installer from vim.org
# Option 2: Via Windows Package Manager
winget install vim.vim
# Option 3: Neovim (recommended)
winget install Neovim.Neovim

# Verify installation
vim --version
# Should show: VIM - Vi IMproved 9.x

# Open Vim (just type vim in terminal)
vim
```

Your First Vim Session: Surviving the Editor

Every Vim beginner has the same terrifying experience: they open Vim by accident and can't exit. Let's fix that right now.

```
# Open a file in Vim
vim myfile.txt

# You are now in NORMAL mode (the default)
# You cannot type text yet — that's intentional!
```

```
WHAT YOU SEE WHEN VIM OPENS:
═══════════════════════════════════════════════════════════


┌──────────────────────────────────────────────────────────┐
│                                                          │
│                                                          │
│~                                                         │
│~                                                         │
│~                                                         │
│~                Vim is ready and waiting.                │
│~            You are in NORMAL mode right now.            │
│~                                                         │
│~                                                         │
│"myfile.txt" [New File]                                   │
└──────────────────────────────────────────────────────────┘
  ↑
  The ~ symbols mean "empty line" (no content yet)
  The status bar at the bottom shows file info
```

The Only 5 Commands You Need to Survive Today

```
ESSENTIAL SURVIVAL COMMANDS
═══════════════════════════════════════════════════════════

To EXIT Vim:
  :q        → Quit (if no changes)
  :q!       → Quit WITHOUT saving (force quit — use when stuck)
  :wq       → Save and quit
  ZZ        → Save and quit (shortcut)
  ZQ        → Quit without saving (shortcut)

To enter INSERT mode (to type text):
  i         → Insert before cursor

To go BACK to Normal mode (from anywhere):
  Esc       → Return to Normal mode (press this when confused!)
  Ctrl+c    → Also returns to Normal mode

To SAVE:
  :w        → Write (save) the file

MEMORIZE THIS FLOW:
  Open Vim → Press 'i' → Type text → Press Esc → Type ':wq' → Enter
```

Hands-on Exercise 1: First Steps

```
# Open a new file
vim exercise1.txt

# Step 1: Press 'i' to enter Insert mode
# Step 2: Type: "Hello, Vim! I am learning to use you."
# Step 3: Press Esc to return to Normal mode
# Step 4: Type ':w' and press Enter to save
# Step 5: Type ':q' and press Enter to quit
# Step 6: Re-open the file: vim exercise1.txt
# Step 7: Verify your text is there
# Step 8: Exit with ZQ (without saving)
```

Section 1.1

Vim's Modal Architecture

🎯 Learning Objectives

- Understand why Vim uses modes (and why it's brilliant, not frustrating)
- Know all major Vim modes and when to use each
- Switch fluently between modes

- Understand why Normal mode is the "home base"

The Modal Philosophy

The most important concept in Vim is **modes**. Vim has different modes, and what you type does different things depending on which mode you're in.

This sounds confusing at first. But here's the insight: **you spend most of your time editing text, not inserting it.** Think about it — when you write code, you:

- Navigate to a line (moving, not typing)
- Delete a word (deleting, not typing)
- Copy a section (copying, not typing)
- Change a variable name (finding+replacing, not typing)
- **Then** maybe type some new code

A regular editor treats the keyboard as a typewriter: every key types a character. To navigate or delete, you hold Ctrl or reach for the mouse.

Vim treats the keyboard as a control panel. In Normal mode, every key is a powerful command. You "switch gears" to Insert mode only when you need to type.

```
THE MODAL ADVANTAGE
═══════════════════════════════════════════════════════════════

REGULAR EDITOR: Moving 5 words right
  → Ctrl+Right, Ctrl+Right, Ctrl+Right, Ctrl+Right, Ctrl+Right
  → OR: reach for mouse, click
  → 5 awkward key combos or hand leaves keyboard

VIM: Moving 5 words right (Normal mode)
  → 5w
  → 2 keystrokes, hands never leave home row

REGULAR EDITOR: Delete from cursor to end of line
  → Shift+End, Delete
  → 2 keys, but disruptive

VIM: Delete from cursor to end of line
  → D
  → 1 key

REGULAR EDITOR: Change the word under cursor
  → Double-click to select, then type replacement
  → Requires mouse

VIM: Change word under cursor
  → ciw (change inside word) then type replacement
  → Pure keyboard, lightning fast
```

The Major Modes

```
VIM MODES OVERVIEW
═══════════════════════════════════════════════════════════════

                  ┌──────────────────────┐
                  │      NORMAL MODE     │
                  │   (The Home Base)    │
                  │                      │
                  │  Navigate, delete,   │
                  │  copy, paste, search │
                  │  Run commands        │
                  └──────────────────────┘
                      ↑  │   ↑   │
             Esc / Ctrl+C  │   │   │
                      │  ↓   │   ↓
          ┌──────────────────┐ ┌──────────────────┐
          │   INSERT MODE    │ │   VISUAL MODE    │
          │  (Typing mode)   │ │ (Selection mode) │
          │                  │ │                  │
          │  i, a, o, I,     │ │  v (char)        │
          │  A, O, s, S, c   │ │  V (line)        │
          └──────────────────┘ │  Ctrl+v (block)  │
                               └──────────────────┘
                      │
                      ↓
          ┌──────────────────────────────────────┐
          │          COMMAND-LINE MODE           │
          │        (Running Ex commands)         │
          │                                      │
```

```
            │  :  → Enter command (:w, :q, :s//)   │
            │  /  → Search forward                 │
            │  ?  → Search backward                │
            └──────────────────────────────────────┘
```

Normal Mode — Your Home

Normal mode is where you spend most of your time. Every character on the keyboard is a command.

```
WHAT KEYS DO IN NORMAL MODE (SAMPLE):
═══════════════════════════════════════════════════════


h j k l    → Move left, down, up, right (arrow keys)
w b e      → Move by word (forward, back, end)
0 $        → Start / end of line
gg G       → Start / end of file
d          → Delete (operator — needs a motion)
y          → Yank/copy (operator — needs a motion)
c          → Change (delete + enter insert mode)
p P        → Paste after / before cursor
u          → Undo
Ctrl+r     → Redo
/          → Search
:          → Enter command-line mode
```

Insert Mode — Typing Text

Insert mode is where you type new text. It behaves like a normal editor.

```
# Ways to enter Insert mode from Normal mode:
i     # Insert BEFORE cursor
a     # Append AFTER cursor
I     # Insert at BEGINNING of line
A     # Append at END of line
o     # Open new line BELOW and insert
O     # Open new line ABOVE and insert
s     # Delete character under cursor and insert (substitute)
S     # Delete entire line and insert
cc    # Same as S — clear line and insert
C     # Delete from cursor to end of line and insert

# Exit Insert mode:
Esc        # Return to Normal mode (most common)
Ctrl+c     # Return to Normal mode (faster but skips abbreviations)
Ctrl+[     # Same as Esc (useful on keyboards without Esc key)
```

```
INSERT MODE INDICATOR:
═══════════════════════════════════════════════════════════


┌──────────────────────────────────────────────────────────┐
│ Hello, I am typing text in Insert mode.                  │
│ _                                                        │
│~                                                         │
│~                                                         │
│-- INSERT --                                              │
└──────────────────────────────────────────────────────────┘
  ↑
  Status bar shows "-- INSERT --" when in Insert mode
  Always check the status bar if confused about which mode you're in!
```

Visual Mode — Selecting Text

Visual mode lets you select text visually, then operate on the selection.

```
# Entering Visual mode:
v        # Character-wise visual (select character by character)
V        # Line-wise visual (select whole lines)
Ctrl+v   # Block visual (select a rectangle of text — super powerful!)

# Once in Visual mode:
# Use any motion key to extend the selection (h j k l w b $ etc.)
# Then press an operator (d to delete, y to copy, c to change, etc.)
```

```
VISUAL MODE EXAMPLE:
═══════════════════════════════════════════════════════════

Text:   function hello() {
        │   return "world";
        │}

After pressing V (line visual) and selecting 3 lines:

        ▓function hello() {▓  ← selected (highlighted)
        ▓    return "world";▓
        ▓}▓

        Press d → delete all 3 lines
        Press y → copy all 3 lines
        Press > → indent all 3 lines

BLOCK VISUAL (Ctrl+v):

        name:  Alice
        name:  Bob
        name:  Carlos

        Ctrl+v, select the "name" column, press I, type "first_"
        → Inserts "first_" before "name" on ALL selected lines simultaneously!

        first_name:  Alice
        first_name:  Bob
        first_name:  Carlos
```

Command-Line Mode

Accessed with `:`, `/`, or `?`. Used for file operations, search/replace, settings.

```
:w          → Save
:q          → Quit
:e file     → Open a file
:s/old/new  → Substitute (find & replace)
:set nu     → Show line numbers
/pattern    → Search forward
?pattern    → Search backward
```

Mode Awareness: The Status Bar

```
READING THE VIM STATUS BAR
═══════════════════════════════════════════════════════════


┌──────────────────────────────────────────────────────────┐
│ [file content]                                           │
│                                                          │
│ filename.txt         [+]    50%    25,10    All         │
│ ──────────────────────────────────────────────────       │
│ -- INSERT --                                             │
└──────────────────────────────────────────────────────────┘


Left section:   filename.txt  → Current file name
                [+]           → File has unsaved changes
Middle section: 50%           → Cursor is 50% through the file
Right section:  25,10         → Line 25, Column 10
                All           → Entire file visible (or Top/Bot/xx%)

Mode indicator (bottom left):
  (nothing)     → Normal mode
  -- INSERT --  → Insert mode
  -- VISUAL --  → Visual mode
  -- V-LINE --  → Visual Line mode
  -- V-BLOCK -- → Visual Block mode
```

Common Mistakes ⚠️

- **Typing commands in Insert mode** — If `hjkl` is typing letters instead of moving cursor, press `Esc` first
- **Not knowing your current mode** — Always check the status bar at the bottom

- **Pressing Esc multiple times** — One `Esc` is always enough to return to Normal mode
- **Panicking** — If lost, press `Esc` twice, then `:q!` to exit without saving and start fresh

---

Section 1.2

Navigation: Moving at the Speed of Thought

🎯 Learning Objectives

- Move efficiently without arrow keys
- Jump to any position in a file in 2-3 keystrokes
- Use word, sentence, paragraph, and file-level navigation
- Understand counts and motion combinations

The Navigation Philosophy

Vim's navigation is built around one principle: **never move more than you need to, never fewer.** Every navigation command targets exactly where you want to go. You'll learn to think "I want to go to the next `{`" and press `]}` — not hold the arrow key for 3 seconds.

```
VIM NAVIGATION HIERARCHY
═══════════════════════════════════════════════════════════

Level 1: CHARACTER movement (h j k l)
  Use when: Moving 1-3 characters

Level 2: WORD movement (w b e W B E)
  Use when: Moving within a line

Level 3: LINE movement (0 ^ $ n% G gg)
  Use when: Jumping across a line or to specific line

Level 4: SCREEN movement (H M L Ctrl+f Ctrl+b)
  Use when: Scrolling or jumping to top/middle/bottom of screen

Level 5: FILE movement (gg G /pattern ` marks)
  Use when: Jumping far across the file

RULE: Always use the highest applicable level.
  Pressing 'j' 20 times = wrong. Using '20j' or '/function' = right.
```

Character Movement: hjkl

The most basic movement — left, down, up, right. These replace arrow keys.

```
     k (up)
      |
h ────┼──── l (right)
(left)│
     j (down)

WHY NOT ARROW KEYS?
  Arrow keys require moving your hand to the right section of the keyboard.
  hjkl are right on the home row — your fingers never move!

  After a few days of practice, hjkl becomes completely natural.
```

```
# In Normal mode:
h    # Move left  (← arrow)
j    # Move down  (↓ arrow)
k    # Move up    (↑ arrow)
l    # Move right (→ arrow)

# With counts (number before any motion = repeat N times):
5j   # Move down 5 lines
3h   # Move left 3 characters
10l  # Move right 10 characters
```

Word Movement

```
# Lowercase: treats punctuation as word boundaries
w    # Forward to start of NEXT word      "hello.world" → 3 words
b    # Backward to start of PREVIOUS word
e    # Forward to END of current/next word
ge   # Backward to END of previous word

# Uppercase: treats ONLY whitespace as boundaries
W    # Forward to start of next WORD      "hello.world" → 1 WORD
B    # Backward to start of previous WORD
E    # Forward to end of current/next WORD

# With counts:
3w   # Move forward 3 words
2b   # Move backward 2 words
```

```
WORD vs WORD EXAMPLES:
═══════════════════════════════════════════════════════════

Text:    hello_world.foo-bar baz

With w (word): hello _ world . foo - bar   baz
               ─────┬─────┬─┬───┬─┬───┬─┬─────
               word1 word2 3  4  5  6  7  8

With W (WORD): hello_world.foo-bar            baz
               ──────────────────────────────
               WORD 1                         WORD 2

Use w for fine-grained navigation within code symbols.
Use W for quick jumps across space-separated tokens.
```

Line Navigation

```
0      # Move to FIRST CHARACTER of line (column 0)
^      # Move to FIRST NON-BLANK character of line
$      # Move to LAST CHARACTER of line
g_     # Move to last non-blank character
+      # Move to first non-blank of NEXT line
-      # Move to first non-blank of PREVIOUS line

# Jump to a specific line:
:25    # Go to line 25 (Command-line mode)
25G    # Go to line 25 (Normal mode)
25gg   # Same as 25G

# By percentage:
50%    # Go to middle of file (50% through)
```

Find Characters on a Line

```
f{char}   # Find: jump forward TO next occurrence of char on this line
F{char}   # Find backward to previous occurrence of char
t{char}   # Till: jump forward to character BEFORE char
T{char}   # Till backward
;         # Repeat last f/F/t/T in same direction
,         # Repeat last f/F/t/T in opposite direction

# Example:
# Line:  "function calculateTax(amount, rate) {"
# Press: fT  → cursor jumps to 'T' in 'Tax'
# Press: ;   → cursor jumps to 'T' in 'Tax' again (next T — but none here, so it stops)
# Press: fa  → cursor jumps to 'a' in 'Tax'
# Press: ;   → cursor jumps to 'a' in 'amount'
# Press: ;   → cursor jumps to 'a' in 'rate'
```

Screen and File Navigation

```
# SCREEN navigation:
H      # High — top of screen
M      # Middle — middle of screen
L      # Low — bottom of screen

# SCROLLING:
Ctrl+f  # Forward (down) one full screen
Ctrl+b  # Backward (up) one full screen
Ctrl+d  # Down half screen
Ctrl+u  # Up half screen
Ctrl+e  # Scroll screen down 1 line (cursor may move)
Ctrl+y  # Scroll screen up 1 line

# FILE navigation:
gg     # Go to FIRST line of file
G      # Go to LAST line of file
50%    # Go to 50% through file

# Center / reposition screen:
zz     # Redraw with cursor at CENTER of screen
zt     # Redraw with cursor at TOP of screen
zb     # Redraw with cursor at BOTTOM of screen
```

Marks — Bookmarks for Your File

```
# Set a mark:
ma     # Mark current position as 'a' (any lowercase letter)
mB     # Mark current position as 'B' (uppercase = global mark, cross-file!)

# Jump to a mark:
'a     # Jump to line of mark 'a' (first non-blank of that line)
`a     # Jump to EXACT position of mark 'a' (line AND column)

# Special marks (set automatically):
''     # Jump back to last jump position
`.     # Jump to last change position
`^     # Jump to last insert position
`[     # Jump to start of last changed/yanked text
`]     # Jump to end of last changed/yanked text

# View all marks:
:marks
```

The Jump List

```
# Vim remembers your jump history
Ctrl+o    # Jump BACK to previous position (Older)
Ctrl+i    # Jump FORWARD in jump history (newer — "In" = forward)

# View jump list:
:jumps
# Shows all positions you've jumped to/from
```

```
JUMP LIST EXAMPLE:
═══════════════════════════════════════════════════════════

You're on line 100. You jump to line 450 (using G or /search).
You jump to line 230.
You jump to line 780.

Ctrl+o → Back to line 230
Ctrl+o → Back to line 450
Ctrl+o → Back to line 100
Ctrl+i → Forward to line 450
```

The `*` and `#` Search Shortcuts

```
*      # Search forward for word under cursor
#      # Search backward for word under cursor
g*     # Search forward (partial match — not whole word only)
g#     # Search backward (partial match)

# Pro Tip: Place cursor on a variable name, press *, and instantly
# jump to all occurrences of that variable!
```

Complete Navigation Reference

```
NAVIGATION CHEAT SHEET
═══════════════════════════════════════════════════════════

CHARACTER:    h(←) j(↓) k(↑) l(→)

WORD:         w(next word)  W(next WORD)  b(prev word)  B(prev WORD)
              e(end word)   E(end WORD)   ge(end prev)

LINE:         0(col 0)  ^(first char)  $(end)  g_(last non-blank)
              f{c}(find char)  t{c}(till char)  ;(repeat) ,(reverse)

VERTICAL:     gg(top file)  G(end file)  nG(line n)  n%(percent)
              H(screen top)  M(screen mid)  L(screen bottom)

SCROLL:       Ctrl+f(page down)  Ctrl+b(page up)
              Ctrl+d(half down)  Ctrl+u(half up)

MARKS:        m{c}(set mark)  '{c}(jump line)  `{c}(jump exact)
              ''(last jump)   Ctrl+o(back)      Ctrl+i(forward)

SEARCH:       /(forward)  ?(backward)  n(next)  N(prev)
              *(word forward)  #(word backward)
```

Hands-on Exercise 2: Navigation Mastery

```
# Create a practice file
vim navigation-practice.txt
```

```
# Paste this content (press i, paste, press Esc)

The quick brown fox jumps over the lazy dog.
Pack my box with five dozen liquor jugs.
How vexingly quick daft zebras jump!
The five boxing wizards jump quickly.
Sphinx of black quartz, judge my vow.
function calculateTotal(price, taxRate) {
    const tax = price * taxRate;
    const total = price + tax;
    return total;
}
function greetUser(name) {
    return "Hello, " + name + "!";
}
```

```
# NAVIGATION CHALLENGES:
# (start at the top of the file with 'gg')

# 1. Jump to line 6:                6G
# 2. Jump to end of file:           G
# 3. Jump to start of file:         gg
# 4. Move down 3 lines:             3j
# 5. Move to end of current line:   $
# 6. Move to start of line:         0
# 7. Move forward 3 words:          3w
# 8. Find 'f' character on line:    ff
# 9. Jump to line with 'greetUser':  /greetUser Enter
# 10. Return to previous position:   Ctrl+o
# 11. Set a mark at current line:    ma
# 12. Jump to first line:            gg
# 13. Return to mark:                'a

# TIME YOURSELF: How long to navigate from line 1 to "greetUser" without arrow keys?
# Goal: Under 3 seconds after practice.
```

Section 1.3

Inserting, Deleting & Basic Editing

🎯 Learning Objectives

- Use all insert mode entry points strategically
- Delete text at character, word, and line levels
- Copy (yank) and paste text efficiently
- Undo and redo changes

Entering Insert Mode: Choosing the Right Entry Point

Each insert command positions the cursor differently — choosing the right one saves keystrokes.

```
INSERT MODE ENTRY POINTS
═══════════════════════════════════════════════════════════

For the line:  "    function hello() {"
                    ^cursor here

i   → Insert BEFORE cursor:          "    fun|ction hello() {"
a   → Append AFTER cursor:           "    func|tion hello() {"
I   → Insert at line BEGINNING:      "|    function hello() {"
A   → Append at line END:            "    function hello() {|"
o   → New line BELOW, insert:        "    function hello() {\n|"
O   → New line ABOVE, insert:        "|\n    function hello() {"
s   → Delete char, insert:           "    fun|tion hello() {" (deleted 'c')
S   → Delete line, insert:           "|" (entire line gone)
cc  → Same as S
C   → Delete to end of line:         "    fun|" (rest deleted, cursor at end)

GOLDEN RULES:
  To add to end of line:      A  (not $i)
  To add to start of line:    I  (not 0i or ^i)
  To add a line below:        o  (not A Enter)
  To add a line above:        O  (not first moving up)
```

Useful Insert Mode Shortcuts

Even while in Insert mode, these shortcuts help:

```
# While in INSERT mode:
Ctrl+w    # Delete word before cursor (same as backspace a word)
Ctrl+u    # Delete from cursor to start of line
Ctrl+h    # Backspace (same as Backspace key)
Ctrl+j    # Enter/newline
Ctrl+t    # Indent current line (add shiftwidth of spaces)
Ctrl+d    # Dedent current line (remove shiftwidth of spaces)
Ctrl+n    # Autocomplete next match
Ctrl+p    # Autocomplete previous match
Ctrl+r {reg}  # Paste from register (e.g., Ctrl+r " pastes clipboard)
```

Deleting Text

```
# CHARACTER level:
x     # Delete character UNDER cursor (like Delete key)
X     # Delete character BEFORE cursor (like Backspace)
5x    # Delete 5 characters forward

# WORD level:
dw    # Delete to start of next word
db    # Delete to start of previous word
de    # Delete to end of word (including last char)
dW    # Delete to next WORD
diw   # Delete INSIDE word (the whole word, no spaces)
daw   # Delete A Word (word + surrounding spaces)

# LINE level:
dd    # Delete entire current line
5dd   # Delete 5 lines
D     # Delete from cursor to end of line
d$    # Same as D
d0    # Delete from cursor to start of line
d^    # Delete to first non-blank of line

# COMBINED with motion:
d2w   # Delete 2 words forward
d3j   # Delete 3 lines down (current + 2 below)
dfx   # Delete from cursor through next 'x'
dt;   # Delete up to (but not including) next ';'
```

```
DELETION PATTERNS:
═══════════════════════════════════════════════════════════════

Line:     "    const userName = getUserName(userId);"

x         Deletes one character at cursor
dd        Deletes the whole line
D         Deletes from cursor to semicolon at end
dw        Deletes "const " (word + space)
diw       Deletes "const" (just the word)
daw       Deletes "const " (word + trailing space)
dt=       Deletes "const userName " (up to but not including =)
df=       Deletes "const userName =" (up to and including =)
d$        Same as D
d0        Deletes from cursor to beginning of line

NOTE: Everything deleted goes into a register — it's not truly "deleted"!
      You can paste it back with p or P.
```

Yanking (Copying) and Pasting

In Vim, "yank" means copy. The yank commands mirror the delete commands exactly.

```
# YANKING (copying):
yy     # Yank (copy) current line
5yy    # Yank 5 lines
Y      # Yank to end of line (same as y$ in most configs)
yw     # Yank word
yiw    # Yank inner word
yaw    # Yank a word (with surrounding spaces)
y$     # Yank to end of line
y0     # Yank to start of line
yG     # Yank from here to end of file
y2j    # Yank 2 lines down

# PASTING:
p      # Paste AFTER cursor (or below current line for line-wise yanks)
P      # Paste BEFORE cursor (or above current line for line-wise yanks)
3p     # Paste 3 times
gp     # Paste after and move cursor to end of pasted text
```

```
YANK AND PASTE IN ACTION:
═══════════════════════════════════════════════════════════════

Original:
  1: function hello() {
  2:     return "hello";
  3: }
  4:
  5: // Need to duplicate hello function

Cursor on line 1. Press: 3yy  (yank 3 lines)
Move cursor to line 5.  Press: p  (paste after line 5)

Result:
  1: function hello() {
  2:     return "hello";
  3: }
  4:
  5: // Need to duplicate hello function
  6: function hello() {
  7:     return "hello";
  8: }
```

Undo and Redo

```
u         # Undo last change
U         # Undo all changes on current line (rarely used)
Ctrl+r    # Redo (undo the undo)

# Vim's undo is POWERFUL:
# - Unlimited undo history (by default, up to 1000 changes)
# - Vim 7.3+: Persistent undo (survives closing the file!)
# - Undo TREE (not just a linear list — see :help undo-tree)

# Count with undo:
5u        # Undo last 5 changes

# Navigate undo tree (advanced):
:earlier 5m    # Go to state 5 minutes ago
:later 1h      # Go to state 1 hour from now
```

Changing Text: The `c` Operator

The `c` (change) operator deletes and enters Insert mode — the most common editing operation.

```
# c works just like d but enters Insert mode after:
cw    # Change word (delete word, enter insert)
ciw   # Change inner word
caw   # Change a word
c$    # Change to end of line
C     # Same as c$
cc    # Change entire line (same as S)
ci"   # Change inside quotes → delete content between quotes, insert
ci(   # Change inside parentheses
ci{   # Change inside curly braces
ci[   # Change inside square brackets
```

```
THE CHANGE COMMAND IN PRACTICE:
═══════════════════════════════════════════════════════════════

Line:  "const message = 'Hello World';"
                         ^cursor inside 'Hello World'

ci'   → Deletes "Hello World", cursor is between quotes:
        "const message = '';"  ← now in Insert mode
        Type: "Goodbye World"
        Press Esc
        "const message = 'Goodbye World';"

        Entire operation: ci' + "Goodbye World" + Esc
        Compare to mouse: click, double-click, type — more steps!
```

Hands-on Exercise 3: Editing Fundamentals

```
vim editing-practice.js
```

```
// Paste this content, then complete the tasks below

const userNmae = "alice";    // Task 1: Fix typo "userNmae" → "userName"
const AGE = 25;              // Task 2: Change AGE to age (lowercase)
var greeting = "Hi";         // Task 3: Change var to const
                             // Task 4: Change "Hi" to "Hello"
function clalculate(x, y) { // Task 5: Fix typo "clalculate" → "calculate"
    return x + y;
}
// Delete this entire comment line    // Task 6: Delete this line
// And this one too                  // Task 7: Delete this line
const items = ["apple", "banana"];   // Task 8: Add "cherry" to the array
```

```
# SOLUTIONS:
# Task 1: Place cursor on "userNmae", press ciw, type "userName", Esc
# Task 2: Place cursor on "AGE", press gUiw to upper, or viw~  to toggle case
#         OR: press ciw, type "age", Esc
# Task 3: Place cursor on "var", press cw, type "const", Esc
# Task 4: Place cursor inside "Hi", press ci", type "Hello", Esc
# Task 5: Place cursor on "clalculate", press ciw, type "calculate", Esc
# Task 6: Move to comment line, press dd
# Task 7: Move to next comment line, press dd
# Task 8: Place cursor before "]", press i, type ", \"cherry\"", Esc
```

Best Practices: Day 1 Summary

```
KEY HABITS TO BUILD FROM DAY 1
═══════════════════════════════════════════════════════════

1. ESCAPE IS YOUR FRIEND
   → Press Esc whenever unsure. It always returns you to safety.

2. HJKL OVER ARROW KEYS
   → Force yourself. It's uncomfortable for 3 days, then natural forever.

3. READ THE STATUS BAR
   → If Vim behaves strangely, check which mode you're in.

4. dd TO DELETE LINES, yy TO COPY
   → The most common operations in Vim. Memorize these.

5. u TO UNDO — LIBERALLY
   → Don't be afraid to try commands. u always bails you out.

6. :w OFTEN
   → Save constantly. Vim doesn't autosave by default.
   → (Configure autosave in vimrc — covered in Section 2.3)
```

Section 2.0

The Vim Language: Operators, Motions & Text Objects

🎯 Learning Objectives

- Understand Vim's composable "language" structure
- Combine operators with motions to create powerful commands
- Use text objects to operate on semantic units
- Write new Vim commands by combining parts you already know

The Grammar of Vim

Vim's most powerful insight is that its commands form a **grammar** — a composable language where you combine simple pieces into complex operations.

```
VIM'S GRAMMAR:
═══════════════════════════════════════════════════════════════

Structure:  [COUNT] OPERATOR [COUNT] MOTION
         OR [COUNT] OPERATOR TEXT_OBJECT

Examples:
  d  w     → delete word           (operator + motion)
  3  d  w  → delete 3 words        (count + operator + motion)
  d  3  w  → delete 3 words        (operator + count + motion — same!)
  d  i  w  → delete inside word    (operator + text object)
  c  i  "  → change inside quotes  (operator + text object)
  y  a  p  → yank a paragraph      (operator + text object)
  >  3  j  → indent 3 lines down   (operator + count + motion)
  g  u  $  → lowercase to end line (operator + motion)

OPERATORS (the "verbs"):
  d  → Delete
  y  → Yank (copy)
  c  → Change (delete + insert)
  >  → Indent right
  <  → Indent left
  =  → Auto-indent
  gU → Uppercase
  gu → Lowercase
  g~ → Toggle case
  !  → Filter through external command

MOTIONS (the "objects" for operator):
```

```
  Any navigation command works as a motion:
  w e b $ 0 ^ G gg f{c} t{c} /{pattern} etc.
```

Text Objects — The Secret Weapon

Text objects operate on semantic units of text: words, sentences, paragraphs, quoted strings, code blocks.

```
# TEXT OBJECT SYNTAX:
#   i = "inner" (excludes surrounding whitespace/delimiters)
#   a = "a" or "around" (includes surrounding whitespace/delimiters)
#
#   Followed by the object type:
#   w  = word
#   W  = WORD (whitespace-delimited)
#   s  = sentence
#   p  = paragraph
#   "  = double-quoted string
#   '  = single-quoted string
#   `  = backtick-quoted string
#   (  = parentheses  (also: b = "block")
#   )  = same as (
#   {  = curly braces  (also: B)
#   }  = same as {
#   [  = square brackets
#   ]  = same as [
#   <  = angle brackets
#   >  = same as <
#   t  = HTML/XML tag
```

```
TEXT OBJECTS IN PRACTICE:
═══════════════════════════════════════════════════════════

String:  "Hello, World!"
             ^cursor anywhere inside

diw   → Delete: "Hello"      (inner word — just the word)
daw   → Delete: "Hello, "    (a word — word + following space)
di"   → Delete: inside the quotes → leaves ""
da"   → Delete: the entire "Hello, World!" including quotes

Function call:  getUserName(userId, options)
                                ^cursor inside parens

di(   → Delete contents: getUserName()
da(   → Delete including parens: getUserName
ci(   → Change contents: getUserName(|)  then type new args
        Result: getUserName(newArg)

HTML:  <div class="container"><p>Hello</p></div>
                                  ^cursor inside p

dit   → Delete inner tag content: <p></p>
dat   → Delete a tag: removes entire <p>Hello</p>
cit   → Change inside tag: <p>|</p>  then type new content
```

The `.` Repeat Command — Vim's Most Powerful Feature

The `.` (dot) command repeats the last change. This is arguably Vim's single most powerful feature.

```
.    # Repeat last change (insert, delete, substitute, etc.)
```

```
DOT REPEAT IN ACTION:
═══════════════════════════════════════════════════════════════

Problem: You need to add a semicolon to the end of 10 lines.

SLOW WAY: Go to end of each line, press A, type ;, press Esc. Repeat 10x.

FAST WAY WITH DOT:
  1. Go to first line
  2. Press A; Esc  (append semicolon, return to normal)
  3. Go to next line (j)
  4. Press .  (repeat the "append semicolon" change)
  5. j.j.j.j.j.j.j.j.  (9 more lines in 18 keystrokes)

OR EVEN FASTER:
  Use :norm A; to add semicolons to all selected lines at once!

DOT + SEARCH:
  Find all "var " declarations and change to "const ":
  1. /var   Enter  (find first)
  2. cw const Esc  (change word to "const")
  3. n.              (find next, repeat change)
  4. n.n.n.          (repeat for all occurrences)

  Vim's answer to a quick find-and-change workflow!
```

Combining Everything: Real Examples

```
COMMON EDITING SCENARIOS
═══════════════════════════════════════════════════════════

SCENARIO 1: Rename a variable throughout a function
  Code: function calc(prce, qty) { return prce * qty; }
  Task: Fix typo "prce" → "price"

  Steps: /prce Enter    (search for first occurrence)
         ciw price Esc  (change inner word to "price")
         n.             (next occurrence, repeat change)
         n.             (if another occurrence)

SCENARIO 2: Swap function arguments
  Code: function greet(lastName, firstName) {
  Task: Swap the arguments

  Steps: Move cursor to "lastName"
         daw            (delete "lastName, " — the word + following space+comma)
         Move cursor after "firstName"
         p              (paste "lastName, " after "firstName")

         Result: function greet(firstName, lastName) {

SCENARIO 3: Wrap text in quotes
  Code: return hello;
  Task: Make it return "hello";

  Steps: Move cursor to "hello"
         ciw "hello" Esc  (change inner word, type with quotes)
```

```
        OR use vim-surround plugin: ysiw" (yank surround inner word with ")

SCENARIO 4: Comment out a block of code
  Code: (lines 10-15 of JavaScript)
  Task: Comment them all out

  Steps: Go to line 10
         Ctrl+v          (enter block visual)
         5j              (select 6 lines)
         I               (insert at beginning of each line)
         //              (type the comment characters)
         Esc             (applies to all selected lines!)
```

Hands-on Exercise 4: The Vim Language

```
// vim language-exercise.js
// Objective: Refactor this messy code using Vim's language

var x = 10
var y = 20
var z = 30

function Add(a, b){
return a + b
}

function Multiply(a,b){
return a*b
}

var result1 = Add(x,y)
var result2 = Multiply(y,z)
console.log(result1)
console.log(result2)
```

```
# TASKS (try to use Vim's language for each):
# 1. Change all "var" to "const" using: /var Enter, ciw const Esc, then n. for rest
# 2. Lowercase function names "Add" → "add" and "Multiply" → "multiply"
#    Place cursor on capital letter, use ~ to toggle case OR guw to lowercase word
# 3. Add spaces around { in function definitions: "function Add(a, b){"  → "function Add(a, b) {"
#    Use f{ then i<space> Esc
# 4. Add missing semicolons at end of return statements
#    Go to line, use A; Esc, then j. for each subsequent line
# 5. Add spaces after commas in function calls: "Add(x,y)" → "Add(x, y)"
#    f, then a<space> Esc, repeat with .
```

Section 2.1

Searching, Finding & Replacing

🎯 Learning Objectives

- Search forward and backward with `/` and `?`
- Use regular expressions for powerful searches
- Perform find-and-replace with `:substitute`
- Use global commands to transform files

Basic Search

```
# Forward search:
/pattern      # Search forward for "pattern"
              # Press Enter to confirm, n for next, N for previous

# Backward search:
?pattern      # Search backward for "pattern"

# After searching:
n             # Next match (in search direction)
N             # Previous match (opposite direction)
*             # Search for word under cursor (forward)
#             # Search for word under cursor (backward)

# Clear search highlighting:
:noh          # No highlight (turns off temporarily)
:set nohlsearch  # Turn off permanently (put in vimrc)
```

Search Options and Flags

```
# Case sensitivity:
/Pattern       # Case sensitive (exact match)
/pattern/i     # Case insensitive for this search
:set ignorecase  # Always ignore case
:set smartcase   # Ignore case UNLESS search contains uppercase (recommended!)

# Whole word matching:
/\bword\b      # Match "word" but not "keyword" or "wordly"
# OR use * which automatically does whole-word search

# Special regex metacharacters in search:
.              # Any single character
*              # Zero or more of previous char
\+             # One or more of previous char (Vim regex)
\?             # Zero or one of previous char
^              # Start of line
$              # End of line
\w             # Word character [a-zA-Z0-9_]
\d             # Digit [0-9]
\s             # Whitespace
[abc]          # Any of: a, b, or c
[^abc]         # Any character EXCEPT a, b, or c
\(group\)      # Capturing group (for use in replacement)
```

```
# Practical search examples:
/function      # Find all "function" occurrences
/^function     # Find lines STARTING with "function"
/function.*{   # Find "function" followed by anything then "{"
/\d\+          # Find one or more digits
/TODO\|FIXME   # Find "TODO" OR "FIXME" (\| = OR in Vim regex)
```

The Substitute Command: Find and Replace

The `:s` (substitute) command is Vim's find-and-replace. It's extremely powerful.

```
SUBSTITUTE SYNTAX:
═══════════════════════════════════════════════════════════════

:[range]s/pattern/replacement/[flags]

RANGE (optional — defaults to current line):
  (nothing)     Current line only
  %             Entire file
  1,10          Lines 1 through 10
  .,$           From current line to end of file
  'a,'b         From mark a to mark b
  (visual)      Visual selection (use V to select, then :s/)

FLAGS (optional):
  g             Global — replace ALL occurrences on each line (not just first)
  i             Case insensitive
  c             Confirm each replacement (interactive)
  n             Count matches without replacing
  e             Don't error if no match found (useful in scripts)
```

```
# EXAMPLES:

# Replace first "foo" with "bar" on current line:
:s/foo/bar/

# Replace ALL "foo" with "bar" on current line:
:s/foo/bar/g

# Replace ALL "foo" with "bar" in ENTIRE FILE:
:%s/foo/bar/g

# Replace with confirmation (asks y/n for each):
:%s/foo/bar/gc
# (y = yes, n = no, a = all, q = quit, l = last then quit)

# Case-insensitive replace:
:%s/foo/bar/gi

# Replace whole word only:
:%s/\bfoo\b/bar/g

# Replace in lines 5-20:
:5,20s/old/new/g

# Replace in visual selection (select with V first):
:'<,'>s/old/new/g
# (Vim fills in '<,'> automatically when you type : in visual mode)
```

Advanced Substitution Patterns

```
# Using captured groups (\1, \2 refer to \(...\) groups):
# Example: swap "firstName lastName" to "lastName, firstName"
:%s/\(\w\+\) \(\w\+\)/\2, \1/g

# Delete trailing whitespace (common cleanup):
:%s/\s\+$//g
# \s\+ = one or more whitespace, $ = end of line, replacement is empty

# Add semicolon to end of lines that don't have one:
:%s/[^;]$/&;/
# [^;] = any char except ;, $ = end of line
# & in replacement = the matched text (so we add ; after it)

# Convert tabs to spaces:
:%s/\t/    /g    # Replace each tab with 4 spaces

# Remove blank lines:
:g/^$/d    # Global command: find blank lines, delete them

# Number each line:
:%s/^/\=line('.').' '/   # Prepend line number to each line
```

The Global Command `:g`

`:g` (global) runs a command on every line matching a pattern.

```
# SYNTAX: :g/pattern/command

# Delete all lines containing "TODO":
:g/TODO/d

# Delete all blank lines:
:g/^$/d

# Print all lines containing "error":
:g/error/p

# Copy all lines containing "import" to end of file:
:g/import/t$

# Execute normal mode command on matching lines:
:g/function/normal I//
# Prepend // to every line containing "function" (commenting them out)

# INVERSE (:v or :g!): Act on lines NOT matching:
:v/important/d    # Delete all lines that do NOT contain "important"
:g!/import/d      # Same — delete lines without "import"
```

Hands-on Exercise 5: Search and Replace

```
// vim search-exercise.js
// This file has intentional issues to fix with search/replace

var userName = "alice"
var userAge = 25
var userEmail = "alice@example.com"
var userActive = true

function getUserName(){
    return userName
}

function getUserAge(){
    return userAge
}

function getUserEmail(){
    return userEmail
}
// TODO: add validation
// TODO: add error handling
// FIXME: getUserEmail doesn't validate format
// TODO: write tests
```

```
# TASKS:
# 1. Replace all "var" with "const" throughout the file:
#    :%s/\bvar\b/const/g

# 2. Add semicolons to end of const declarations (lines with "const"):
#    :g/^const/s/$/;/

# 3. Add parentheses content to empty function signatures — add a "req, res" param:
#    :%s/function get\(\w\+\)()/function get\1(req, res)/g

# 4. List all TODO/FIXME lines (don't delete — just see them):
#    :g/TODO\|FIXME/p

# 5. Delete all TODO lines:
#    :g/TODO/d

# 6. After deleting, confirm with: :g/TODO/p (should return no results)
```

Section 2.2

Registers, Macros & Automation

🎯 Learning Objectives

- Understand Vim's register system for multiple clipboards
- Record and play back macros for repetitive tasks
- Use special registers for system clipboard, last search, etc.
- Build complex macros for real editing automation

Registers: Multiple Clipboards

Vim doesn't have just one clipboard — it has **26 named registers** (a-z), plus special registers.

```
VIM'S REGISTER SYSTEM
═══════════════════════════════════════════════════════════════

Named registers:    "a through "z
                    26 independent clipboards!

Special registers:
  ""  → Unnamed (default) — last yank or delete goes here
  "0  → Yank register — ONLY yanks (not deletes) go here
  "1  → Last delete (of > 1 line)
  "2  → Second-to-last delete... and so on to "9
  "+  → System clipboard (the one Ctrl+C/Ctrl+V use)
  "*  → Primary selection (Linux X11 middle-click)
  "=  → Expression register (evaluate Vim expressions)
  "/  → Last search pattern
  ":  → Last Ex command
  "%  → Current filename
  "#  → Alternate filename
  ".  → Last inserted text
```

```
# USING REGISTERS:

# Yank into a named register:
"ayy    # Yank current line into register 'a'
"byw    # Yank current word into register 'b'
"a3yy   # Yank 3 lines into register 'a'

# Delete into a named register:
"adw    # Delete word, save in register 'a'
"add    # Delete line, save in register 'a'

# Paste from a register:
"ap     # Paste from register 'a'
"bp     # Paste from register 'b'

# Paste from system clipboard:
"+p     # Paste from system clipboard (OS clipboard)
"*p     # Paste from primary selection (Linux)

# Copy TO system clipboard:
"+yy    # Yank line to system clipboard
"+y3j   # Yank 3 lines down to system clipboard

# View all registers:
:reg           # Show all register contents
:reg a b c     # Show registers a, b, and c only
```

```
REGISTER WORKFLOW EXAMPLE:
═══════════════════════════════════════════════════════════

Task: Move 3 blocks of code to different locations

1. Yank block 1 into register 'a':
   V (select lines), "ay  (yank into 'a')

2. Yank block 2 into register 'b':
   V (select lines), "by  (yank into 'b')

3. Yank block 3 into register 'c':
   V (select lines), "cy  (yank into 'c')

4. Navigate to destination 1, paste: "ap
5. Navigate to destination 2, paste: "bp
6. Navigate to destination 3, paste: "cp

Without named registers, you'd have to paste, navigate back,
yank the next block, navigate to destination... multiple round trips!
```

Macros: Recording and Replaying

A macro records a sequence of keystrokes and lets you replay them. Perfect for repetitive edits.

```
# RECORDING A MACRO:
q{register}    # Start recording into register (a-z)
               # Do your editing...
q              # Stop recording

# PLAYING BACK:
@{register}    # Play macro from register
@@             # Replay the last-run macro
5@a            # Play macro 'a' 5 times
100@a          # Play macro 'a' 100 times (until it fails)
```

```
MACRO WORKFLOW — STEP BY STEP EXAMPLE
═══════════════════════════════════════════════════════════════

PROBLEM: You have 20 lines like this that need reformatting:
  alice,25,engineer
  bob,30,designer
  carol,28,manager

TARGET FORMAT:
  { name: "alice", age: 25, role: "engineer" }
  { name: "bob", age: 30, role: "designer" }
  { name: "carol", age: 28, role: "manager" }

SOLUTION WITH MACRO:

Step 1: Place cursor at beginning of first line "alice,25,engineer"

Step 2: Start recording into register 'a':
  qa

Step 3: Perform the transformation on THIS line:
  I{ name: "   → Insert prefix
  Esc
  f,            → Find the first comma
  s", age:      → Replace comma with ", age:
  Esc
  f,            → Find second comma
  s, role: "   → Replace with ", role: "
  Esc
  A" }          → Append closing
  Esc
```

```
  j             → Move to next line (IMPORTANT for macro looping!)
  0             → Go to start of line (prepare for next iteration)

Step 4: Stop recording:
  q

Step 5: Replay for remaining 19 lines:
  19@a

DONE! All 20 lines are transformed.
```

Writing Robust Macros

```
MACRO BEST PRACTICES
═══════════════════════════════════════════════════════════

1. ALWAYS end your macro by moving to the next line (j0 or j^)
   This makes macros repeatable with @@ or N@a

2. USE ABSOLUTE POSITIONING when possible:
   ✅ 0 (go to start of line)
   ✅ $ (go to end of line)
   ✅ f{char} (find specific character)
   ❌ hjkl-based movement (breaks if file content varies)

3. USE TEXT OBJECTS:
   ✅ ciw (change inner word — works regardless of word length)
   ❌ 5x (delete 5 chars — breaks if word length varies)

4. TEST ON ONE LINE FIRST:
   Record on line 1, replay on line 2 with @a before bulk-running

5. EDIT MACROS IN A REGISTER:
   :let @a = substitute(@a, 'old', 'new', 'g')
   OR paste register to buffer: "ap, edit, yank back: "ayy

6. HANDLE ERRORS:
   If a macro errors mid-way, it stops (good behavior)
   Run with count: 100@a (runs until an error, then stops)
```

Advanced Macro Example: Real-World Code Refactoring

```javascript
// BEFORE: Old callback-style JavaScript
fs.readFile('data.json', function(err, data) {
    if (err) throw err;
    console.log(data);
})
fs.writeFile('out.json', content, function(err) {
    if (err) throw err;
    console.log('done');
})
fs.mkdir('output', function(err) {
    if (err) throw err;
    console.log('dir created');
})
```

```
# MACRO to convert "function(err," to "async/await" pattern:

# Step 1: Record macro 'a'
qa

# Step 2: Transform first occurrence
# (cursor at start of line with fs.readFile etc.)
$               # Go to end of line
F,              # Find last comma before "function"
dt)             # Delete to closing paren
a               # Append (we're removing the callback arg)
# ... (this gets complex — in practice, use :s for this type of change)
q

# More practical: Use :s for this pattern
:%s/function(err, \(\w\+\))/async (\1) =>/g
```

Hands-on Exercise 6: Macros

```
Create a file: vim macro-exercise.csv

name,salary,department
alice,75000,engineering
bob,65000,marketing
carol,80000,engineering
dave,72000,sales
eve,68000,marketing
```

```
# TASK: Transform CSV into JavaScript objects using a macro

# TARGET OUTPUT:
# { name: "alice", salary: 75000, department: "engineering" },
# { name: "bob", salary: 65000, department: "marketing" },
# ... (for all 5 data rows)

# STEPS:
# 1. Move cursor to line 2 (alice line)
# 2. Start recording: qa
# 3. Transform the line:
#    I{ name: "   → insert opening
#    f, → find first comma
#    s", salary:   → replace comma
#    f, → find second comma
#    s, department: "  → replace comma
#    A" },   → append closing
#    Esc
#    j0  → move to next line
# 4. Stop recording: q
# 5. Run for remaining 4 lines: 4@a
# 6. Delete the CSV header line: gg dd
```

Section 2.3

Vim Configuration & the vimrc

🎯 Learning Objectives

- Understand the vimrc file and how settings work
- Configure essential settings for a productive environment
- Set up syntax highlighting, line numbers, and indentation
- Create custom key mappings and abbreviations

The vimrc File

The vimrc is Vim's configuration file. It runs every time Vim starts.

```
# Location:
~/.vimrc          # Vim (on Linux/Mac)
~/.config/nvim/init.vim  # Neovim (or init.lua for Lua config)

# Create/edit vimrc:
vim ~/.vimrc
```

The Essential vimrc — Start Here

```
" ═══════════════════════════════════════════════════════════════
" ESSENTIAL VIMRC — A beginner's starting configuration
" Lines starting with " are comments
" ═══════════════════════════════════════════════════════════════

" ── ENCODING ──────────────────────────────────────────────
set encoding=utf-8          " Use UTF-8 encoding

" ── APPEARANCE ────────────────────────────────────────────
syntax enable               " Enable syntax highlighting
set number                  " Show line numbers
set relativenumber          " Show RELATIVE line numbers (great for motions!)
set cursorline              " Highlight the current line
set colorcolumn=80          " Show a column guide at 80 characters
set scrolloff=8             " Keep 8 lines above/below cursor when scrolling
set wrap                    " Wrap long lines
set linebreak               " Break lines at word boundaries (not mid-word)
set showmatch               " Highlight matching brackets

" ── SEARCH ────────────────────────────────────────────────
set hlsearch                " Highlight search results
set incsearch               " Show matches as you type (incremental search)
set ignorecase              " Case insensitive search by default
set smartcase               " BUT case sensitive if pattern has uppercase

" ── INDENTATION ───────────────────────────────────────────
set expandtab               " Use spaces instead of tabs
set tabstop=4               " Tab width = 4 spaces
set shiftwidth=4            " Indent width = 4 spaces
```

```
set softtabstop=4           " Backspace deletes 4 spaces at once
set autoindent              " Automatically indent new lines
set smartindent             " Smart auto-indenting for code

" ── BEHAVIOR ──────────────────────────────────────────────
set nocompatible            " Don't try to be Vi-compatible (use full Vim)
set backspace=indent,eol,start  " Allow backspace over everything
set hidden                  " Allow switching buffers without saving
set nowrap                  " Don't wrap lines (override above if preferred)
set mouse=a                 " Enable mouse support (controversial, but helpful for beginners)
set clipboard=unnamedplus   " Use system clipboard for y/p operations
set undofile                " Persistent undo (survives closing Vim!)
set undodir=~/.vim/undodir  " Where to store undo files

" ── STATUS BAR ────────────────────────────────────────────
set laststatus=2            " Always show status bar
set showmode                " Show current mode in status bar
set showcmd                 " Show incomplete commands in status bar
set ruler                   " Show cursor position in status bar
set wildmenu                " Enhanced command-line completion

" ── PERFORMANCE ───────────────────────────────────────────
set lazyredraw              " Don't redraw during macros (faster)
set ttyfast                 " Faster terminal connection

" ── FILE HANDLING ─────────────────────────────────────────
set autoread                " Reload files changed outside Vim
set nobackup                " Don't create backup files
set noswapfile              " Don't create swap files
" Alternative: keep backups in a central location:
" set backupdir=~/.vim/backups
" set directory=~/.vim/swaps
```

```
" ── SPLIT BEHAVIOR ──────────────────────────────────────────
set splitbelow              " Horizontal splits open below
set splitright              " Vertical splits open to the right

" ── FILETYPE ────────────────────────────────────────────────
filetype on                 " Enable filetype detection
filetype plugin on          " Enable filetype-specific plugins
filetype indent on          " Enable filetype-specific indentation
```

Key Mappings — The Power of Customization

```
" ═══════════════════════════════════════════════════════════════
" KEY MAPPINGS
" Format: {mode}map {keys} {action}
" Modes: n=normal, i=insert, v=visual, c=command
" Use 'nore' (no-remap) versions to avoid recursive mappings!
" ═══════════════════════════════════════════════════════════════

" Set the leader key (\ by default — <Space> is popular)
let mapleader = " "

" ── ESSENTIAL MAPPINGS ──────────────────────────────────────

" Clear search highlighting with Escape
nnoremap <Esc> :noh<CR><Esc>

" Save with Ctrl+S (like most editors)
nnoremap <C-s> :w<CR>
inoremap <C-s> <Esc>:w<CR>a

" Quit with leader+q
nnoremap <leader>q :q<CR>
nnoremap <leader>Q :q!<CR>

" Source vimrc (reload config) with leader+r
nnoremap <leader>r :source ~/.vimrc<CR>

" Edit vimrc quickly
nnoremap <leader>ev :e ~/.vimrc<CR>
```

```
" ── NAVIGATION MAPPINGS ──────────────────────────────────────

" Keep cursor centered when jumping through search results
nnoremap n nzzzv
nnoremap N Nzzzv

" Move lines up/down with Alt+j/k (like most modern editors)
nnoremap <A-j> :m .+1<CR>==
nnoremap <A-k> :m .-2<CR>==
inoremap <A-j> <Esc>:m .+1<CR>==gi
inoremap <A-k> <Esc>:m .-2<CR>==gi
vnoremap <A-j> :m '>+1<CR>gv=gv
vnoremap <A-k> :m '<-2<CR>gv=gv

" Navigate between windows with Ctrl+hjkl
nnoremap <C-h> <C-w>h
nnoremap <C-j> <C-w>j
nnoremap <C-k> <C-w>k
nnoremap <C-l> <C-w>l

" ── EDITING MAPPINGS ─────────────────────────────────────────

" Better indenting in visual mode (stay in visual after indent)
vnoremap < <gv
vnoremap > >gv

" Select all
nnoremap <C-a> ggVG

" Duplicate line with leader+d
nnoremap <leader>d yyp
```

```
" ── BUFFER MAPPINGS ──────────────────────────────────────────
nnoremap <leader>bn :bnext<CR>    " Next buffer
nnoremap <leader>bp :bprev<CR>    " Previous buffer
nnoremap <leader>bd :bd<CR>       " Delete (close) buffer
nnoremap <leader>bl :ls<CR>       " List buffers
```

Abbreviations

```
" ═══════════════════════════════════════════════════════════════
" ABBREVIATIONS
" Expand short text into longer text in Insert mode
" ═══════════════════════════════════════════════════════════════

" Common typo corrections:
iabbrev teh the
iabbrev recieve receive
iabbrev adn and

" Code snippets:
iabbrev ;fn function () {<CR>}<Esc>O   " ;fn → function body
iabbrev ;cl console.log();<Esc>F)i     " ;cl → console.log() with cursor inside
iabbrev ;if if () {<CR>}<Esc>k$F)i     " ;if → if statement

" Your info:
iabbrev @@e your.email@example.com
iabbrev @@n Your Full Name
```

Colorschemes

```
" Set a colorscheme (Vim comes with several built-in):
colorscheme desert      " Dark scheme, good built-in choice
colorscheme elflord     " Another built-in dark scheme
colorscheme morning     " Light scheme

" Check available schemes:
" :colorscheme <Tab>  (tab-completion shows all options)

" For popular themes, install via plugin manager (see Section 3.2):
" Plugin 'morhetz/gruvbox'       → colorscheme gruvbox
" Plugin 'dracula/vim'           → colorscheme dracula
" Plugin 'joshdick/onedark.vim'  → colorscheme onedark

" Enable 24-bit color (if your terminal supports it):
set termguicolors
```

Per-Filetype Settings

```
" ═══════════════════════════════════════════════════════════════
" FILETYPE-SPECIFIC SETTINGS
" ═══════════════════════════════════════════════════════════════

" Using autocmd to apply settings to specific file types:
autocmd FileType python setlocal tabstop=4 shiftwidth=4 expandtab
autocmd FileType javascript setlocal tabstop=2 shiftwidth=2 expandtab
autocmd FileType html setlocal tabstop=2 shiftwidth=2 expandtab
autocmd FileType yaml setlocal tabstop=2 shiftwidth=2 expandtab
autocmd FileType go setlocal tabstop=4 shiftwidth=4 noexpandtab  " Go uses real tabs

" Better: use ftplugin files:
" ~/.vim/ftplugin/python.vim  → settings only for Python
" ~/.vim/ftplugin/js.vim      → settings only for JavaScript
```

Hands-on Exercise 7: Your vimrc

```
# Create and configure your vimrc:
vim ~/.vimrc

# TASKS:
# 1. Copy the essential vimrc from above
# 2. Change tabstop and shiftwidth to your preferred size (2 or 4)
# 3. Add a mapping for <leader>w to save (:w)
# 4. Set your preferred colorscheme
# 5. Add an abbreviation for something you type often
# 6. Source your vimrc: :source ~/.vimrc
# 7. Verify settings work: open a .py file, check indentation
```

Section 3.0

Buffers, Windows, Tabs & the File System

🎯 Learning Objectives

- Understand and manage Vim buffers
- Split windows for side-by-side editing
- Use tabs for workspace organization

- Navigate and manage files with netrw and command-line tools

---

Buffers: Open Files in Memory

A **buffer** is Vim's representation of an open file. You can have many buffers open simultaneously.

```
VIM'S FILE MANAGEMENT LAYERS
═══════════════════════════════════════════════════════════════

BUFFER  → A file loaded into memory (may not be visible)
WINDOW  → A viewport showing a buffer (the pane you see)
TAB     → A collection of windows (a "layout")

  ┌─────────────────────────────────────────────────────────┐
  │ TAB 1                                                   │
  │ ┌──────────────────────┬──────────────────────────────┐ │
  │ │  WINDOW 1            │  WINDOW 2                    │ │
  │ │  (showing buffer A)  │  (showing buffer B)          │ │
  │ │                      │                              │ │
  │ │  app.js              │  styles.css                  │ │
  │ └──────────────────────┴──────────────────────────────┘ │
  └─────────────────────────────────────────────────────────┘

  ┌─────────────────────────────────────────────────────────┐
  │ TAB 2                                                   │
  │ ┌─────────────────────────────────────────────────────┐ │
  │ │  WINDOW 1 (full width)                              │ │
  │ │  (showing buffer C)                                 │ │
  │ │  README.md                                          │ │
  │ └─────────────────────────────────────────────────────┘ │
  └─────────────────────────────────────────────────────────┘

  Also in memory but not displayed:
  Buffer D: tests/app.test.js  (open but not in any window)
  Buffer E: package.json       (open but not in any window)
```

```
# Buffer commands:
:e filename     # Open a file into a new buffer
:ls             # List all open buffers
:ls!            # List all buffers including unlisted

# :ls shows:
#   1 %a   "app.js"            line 1    ← current buffer
#   2  a   "styles.css"        line 1    ← also active (visible)
#   3  h   "tests/app.test.js" line 1    ← hidden buffer
# Flags: % = current, a = active, h = hidden, + = modified

:b 2            # Switch to buffer 2
:b app.js       # Switch to buffer by name (tab-complete works!)
:bn             # Next buffer
:bp             # Previous buffer
:bd             # Delete (close) current buffer
:bd 3           # Delete buffer 3
:bufdo %s/foo/bar/g  # Run substitution on ALL buffers

# Jumping between recently used buffers:
Ctrl+^          # Toggle between current and alternate (#) buffer
                # (fastest way to flip between 2 files!)
```

Windows: Split Views

```
# HORIZONTAL split (stacked top/bottom):
:sp             # Split current buffer horizontally
:sp filename    # Split and open file
Ctrl+w s        # Same: split horizontally

# VERTICAL split (side by side):
:vsp            # Split current buffer vertically
:vsp filename   # Split and open file
Ctrl+w v        # Same: split vertically

# NAVIGATING between windows:
Ctrl+w h        # Move to window on LEFT
Ctrl+w j        # Move to window BELOW
Ctrl+w k        # Move to window ABOVE
Ctrl+w l        # Move to window on RIGHT
Ctrl+w w        # Cycle through windows
Ctrl+w p        # Go to previous (last used) window

# RESIZING windows:
Ctrl+w +        # Increase height
Ctrl+w -        # Decrease height
Ctrl+w >        # Increase width
Ctrl+w <        # Decrease width
Ctrl+w =        # Make all windows equal size
Ctrl+w _        # Maximize height of current window
Ctrl+w |        # Maximize width of current window
5Ctrl+w +       # Increase height by 5 rows
```

```
# CLOSING windows:
Ctrl+w c        # Close current window (buffer stays open)
Ctrl+w o        # Close all OTHER windows (keep only current)
:q              # Close current window (closes Vim if last window)
```

```
SPLIT LAYOUT EXAMPLE:
═══════════════════════════════════════════════════════════════

:vsp tests/app.test.js    → Opens side-by-side

┌────────────────────┬────────────────────────┐
│ app.js             │ tests/app.test.js      │
│                    │                        │
│ function add(a,b){ │ test('add works', () => │
│   return a + b;    │   expect(add(2,3))      │
│ }                  │     .toBe(5);          │
│                    │ });                    │
│                    │                        │
└────────────────────┴────────────────────────┘

Perfect for: Test-driven development, comparing files, docs + code
```

Tabs: Multiple Layouts

```
# TAB commands:
:tabnew              # Open new empty tab
:tabnew filename     # Open file in new tab
:tabn                # Next tab
:tabp                # Previous tab
gt                   # Next tab (Normal mode)
gT                   # Previous tab (Normal mode)
3gt                  # Go to tab 3
:tabclose            # Close current tab
:tabonly             # Close all other tabs
:tabs                # List all tabs

# Moving tabs:
:tabmove 0           # Move tab to first position
:tabmove             # Move tab to last position
```

File Navigation with netrw

Vim's built-in file explorer (no plugin needed!):

```
# Open netrw:
:Explore            # Open in current window
:Sexplore           # Open in horizontal split
:Vexplore           # Open in vertical split
vim .               # Open current directory in Vim

# In netrw:
Enter              # Open file or directory
-                  # Go up a directory
%                  # Create new file
d                  # Create new directory
R                  # Rename file/directory
D                  # Delete file/directory
i                  # Change listing style (thin/long/wide/tree)
```

Hands-on Exercise 8: File Management

```
# Create a project structure to practice with:
mkdir -p ~/vim-practice/{src,tests,docs}
touch ~/vim-practice/src/app.js
touch ~/vim-practice/src/utils.js
touch ~/vim-practice/tests/app.test.js
touch ~/vim-practice/docs/README.md

# Open Vim in the project directory:
cd ~/vim-practice
vim .

# TASKS in netrw:
# 1. Navigate into src/
# 2. Open app.js
# 3. From app.js, open a vertical split: :vsp src/utils.js
# 4. Navigate between splits with Ctrl+w h/l
# 5. Open tests/app.test.js in a horizontal split: :sp tests/app.test.js
# 6. View all open buffers: :ls
# 7. Switch to README.md using :e docs/README.md
# 8. Toggle back to app.js with Ctrl+^
```

Section 3.1

Advanced Editing Techniques

🎯 Learning Objectives

- Use advanced text manipulation techniques
- Work with multiple cursors (via macros and visual block)
- Use external commands from within Vim
- Master indentation, folding, and formatting

Visual Block Mode — Multiple Cursors

Vim's block visual mode (`Ctrl+v`) is incredibly powerful for column-based editing.

```
# Enter block visual mode:
Ctrl+v

# Then select a rectangle, and:
I{text}Esc   # Insert text at START of each selected line
A{text}Esc   # Append text at END of each selected line
r{char}      # Replace each selected character with {char}
d            # Delete the selected block
y            # Yank the selected block
~            # Toggle case of selected block
```

```
VISUAL BLOCK IN ACTION:
═══════════════════════════════════════════════════════════════

EXAMPLE 1: Add "#" comment prefix to multiple lines

Original:
  line one
  line two
  line three

Step 1: Ctrl+v (block visual)
Step 2: Select first column of all 3 lines (2j)
Step 3: I (insert mode for all lines)
Step 4: Type: #
Step 5: Esc

Result:
  # line one
  # line two
  # line three

───────────────────────────────────────────────────────────────

EXAMPLE 2: Delete a column from aligned data

Original:
  alice     25    engineer
  bob       30    designer
  carol     28    manager

Ctrl+v, select the "25    " column across 3 rows, press d
```

```
Result:
  alice     engineer
  bob       designer
  carol     manager
```

Indentation and Formatting

```
# INDENTATION:
>>    # Indent current line right (by shiftwidth)
<<    # Indent current line left
5>>   # Indent 5 lines right
=     # Auto-indent (operator — needs a motion)
==    # Auto-indent current line
=G    # Auto-indent from here to end of file
=ip   # Auto-indent inner paragraph
gg=G  # Auto-indent ENTIRE FILE (most common)

# In VISUAL mode:
> or <     # Indent or dedent selection
>gv        # Indent and re-select (great for repeated indenting)

# FORMATTING TEXT:
gq{motion}   # Format text (wrap to textwidth, e.g., 80 chars)
gqq          # Format current line
gqip         # Format inner paragraph
gqG          # Format from cursor to end of file

# JOINING LINES:
J            # Join current line with the line below (adds space)
gJ           # Join without adding space
3J           # Join current line with next 2 lines
```

Using External Commands

Vim can communicate with your shell and external programs:

```
# RUN a shell command and see output:
:!ls                    # Run ls in shell
:!git status            # Run git status
:!python3 %             # Run the current file with Python
                        # (% = current filename)
:!node %                # Run current file with Node.js

# FILTER selection through an external command:
# (takes selection as stdin, replaces selection with stdout)
:%!sort                 # Sort entire file alphabetically
:%!sort -n              # Sort numerically
:%!column -t            # Align into columns (pretty-print tables)
:.,+5!sort              # Sort current line + next 5 lines

# Useful with selected text:
V5j                     # Select 6 lines
:'<,'>!sort             # Sort them

# INSERT command output into file:
:r !date                # Insert today's date at cursor
:r !ls src/             # Insert directory listing
:r !curl -s api.example.com/data  # Insert API response
```

```
EXTERNAL COMMAND EXAMPLES IN PRACTICE:
═══════════════════════════════════════════════════════════════

SORT import statements:
  ggVG (select all) → :!sort → imports are now alphabetically sorted

PRETTIFY JSON:
  :%!python3 -m json.tool  → formats JSON in current file

ALIGN columns:
  Visual select table data → :'<,'>!column -t → aligned columns

COUNT words in file:
  :!wc -w %   → shows word count without leaving Vim

RUN tests and see output:
  :!npm test  → run tests in terminal output
```

Code Folding

Folding collapses sections of code so you can see the big picture.

```
# FOLD METHODS (set in vimrc):
:set foldmethod=indent   # Fold based on indentation (good for Python)
:set foldmethod=syntax   # Fold based on language syntax (good for most)
:set foldmethod=marker   # Fold between {{{ and }}} markers
:set foldmethod=manual   # Create folds manually

# FOLD COMMANDS:
zo    # Open (unfold) fold under cursor
zc    # Close (fold) fold under cursor
za    # Toggle fold (open if closed, close if open)
zO    # Open fold AND all nested folds
zC    # Close fold AND all nested folds
zR    # Open ALL folds in file
zM    # Close ALL folds in file
zj    # Move to next fold
zk    # Move to previous fold
```

```
FOLDING IN ACTION:
═══════════════════════════════════════════════════════════════

With :set foldmethod=syntax on a JavaScript file:

EXPANDED (zR):
  function getUserProfile(userId) {
      const user = await db.find(userId);
      if (!user) {
          return null;
      }
      return { id: user.id, name: user.name };
  }

  function updateUserProfile(userId, data) {
      // ... 15 more lines
  }

FOLDED (zM):
  +-- function getUserProfile(userId) { --- 7 lines ---
  +-- function updateUserProfile(userId, data) { --- 20 lines ---

Now you can see ALL functions at a glance and expand only what you need.
```

Section 3.2

Plugins & the Modern Vim Ecosystem

🎯 Learning Objectives

- Set up a plugin manager (vim-plug)
- Install and configure essential plugins
- Understand the modern Neovim plugin ecosystem
- Build a professional development environment

Plugin Managers

```
THE VIM PLUGIN ECOSYSTEM
═══════════════════════════════════════════════════════════

Without plugins: Vim is a powerful text editor
With plugins:    Vim becomes a full IDE

Popular plugin managers:
  vim-plug    → Simple, fast, most popular (we'll use this)
  Vundle      → Older, still widely used
  Pathogen    → Minimal, just manages runtimepath
  packer.nvim → Neovim-specific, Lua-based (for Neovim users)
  lazy.nvim   → Modern Neovim, lazy-loading (fastest startup)
```

Installing vim-plug

```
# Install vim-plug (one-time setup):
curl -fLo ~/.vim/autoload/plug.vim --create-dirs \
    https://raw.githubusercontent.com/junegunn/vim-plug/master/plug.vim

# For Neovim:
sh -c 'curl -fLo "${XDG_DATA_HOME:-$HOME/.local/share}"/nvim/site/autoload/plug.vim --create-dirs \
       https://raw.githubusercontent.com/junegunn/vim-plug/master/plug.vim'
```

Configuring Plugins in vimrc

```
" ═══════════════════════════════════════════════════════════════
" PLUGINS (using vim-plug)
" ═══════════════════════════════════════════════════════════════

call plug#begin('~/.vim/plugged')

" ── THEMES ──────────────────────────────────────────────
Plug 'morhetz/gruvbox'              " Gruvbox theme (warm, popular)
Plug 'dracula/vim', { 'as': 'dracula' }  " Dracula theme (dark, popular)

" ── FILE NAVIGATION ─────────────────────────────────────
Plug 'preservim/nerdtree'           " File tree sidebar
Plug 'ctrlpvim/ctrlp.vim'           " Fuzzy file finder (Ctrl+P)
" OR (more modern alternative):
Plug 'junegunn/fzf', { 'do': { -> fzf#install() } }
Plug 'junegunn/fzf.vim'             " FZF integration for Vim

" ── EDITING ENHANCEMENTS ────────────────────────────────
Plug 'tpope/vim-surround'           " Surround text with brackets/quotes
Plug 'tpope/vim-commentary'         " Easy commenting (gc)
Plug 'jiangmiao/auto-pairs'         " Auto-close brackets and quotes
Plug 'easymotion/vim-easymotion'    " Jump to any visible character

" ── GIT INTEGRATION ─────────────────────────────────────
Plug 'tpope/vim-fugitive'           " Git commands from Vim (:Git)
Plug 'airblade/vim-gitgutter'       " Show git diff in the gutter

" ── STATUS BAR ──────────────────────────────────────────
Plug 'vim-airline/vim-airline'      " Beautiful status bar
```

```
Plug 'vim-airline/vim-airline-themes'

" ── CODE COMPLETION & INTELLIGENCE ──────────────────────────
Plug 'neoclide/coc.nvim', {'branch': 'release'}  " VS Code-like completion

" ── SYNTAX HIGHLIGHTING ─────────────────────────────────────
Plug 'sheerun/vim-polyglot'         " Better syntax for 100+ languages

" ── UTILITIES ───────────────────────────────────────────────
Plug 'tpope/vim-repeat'             " Make vim-surround work with .
Plug 'preservim/vim-indent-guides'  " Visual indent guides
Plug 'mbbill/undotree'              " Visual undo history tree

call plug#end()

" ── INSTALL: After adding plugins, run :PlugInstall
" ── UPDATE:  :PlugUpdate
" ── REMOVE:  Delete the Plug line, then :PlugClean
```

The Most Essential Plugins Explained

vim-surround — Change surrounding characters

```
# Add surroundings:
ysiw"     # Surround inner word with double quotes
ysiw(     # Surround inner word with parentheses (with spaces)
ysiw)     # Surround inner word with parentheses (no spaces)
ys$"      # Surround to end of line with quotes
yss{      # Surround entire line with curly braces

# Change surroundings:
cs"'      # Change surrounding " to '
cs'"      # Change surrounding ' to "
cs({      # Change surrounding ( to { (with spaces)
cst"      # Change surrounding HTML tag to "

# Delete surroundings:
ds"       # Delete surrounding double quotes
ds(       # Delete surrounding parentheses
dst       # Delete surrounding HTML tag
```

```
VIM-SURROUND EXAMPLES:
═══════════════════════════════════════════════════════════

"Hello World"   → cs"' →  'Hello World'     (change " to ')
(some text)     → ds( →   some text          (delete parens)
hello           → ysiw" → "hello"            (add quotes)
<p>Hello</p>   → cst<h1> → <h1>Hello</h1>   (change tag)
function(){}    → ysit<div> → wraps with div
```

vim-commentary — Comment/uncomment easily

```
gcc      # Toggle comment on current line
gc3j     # Toggle comment on 3 lines down
gcap     # Toggle comment on entire paragraph
gci{     # Toggle comment inside { block }

# In visual mode:
Vgc      # Comment selected lines
```

NERDTree — File explorer sidebar

```
:NERDTree         # Open file tree
:NERDTreeToggle   # Toggle (open/close)

" In NERDTree:
o         # Open file
t         # Open in new tab
s         # Open in vertical split
i         # Open in horizontal split
m         # Show menu (create, rename, delete files)
R         # Refresh tree
?         # Help
```

fzf.vim — Fuzzy finder (the most impactful plugin)

```
:Files            # Fuzzy find files in project
:GFiles           # Fuzzy find Git-tracked files
:Buffers          # Fuzzy find open buffers
:Lines            # Fuzzy search through all buffer lines
:BLines           # Fuzzy search current buffer
:Ag pattern       # Silver Searcher (grep across project)
:Rg pattern       # ripgrep across project (faster)
:History          # Recently opened files
:Commands         # Fuzzy find Vim commands
:Maps             # Fuzzy find key mappings

" Add to vimrc for shortcuts:
nnoremap <leader>f :Files<CR>
nnoremap <leader>b :Buffers<CR>
nnoremap <leader>g :Rg<CR>
nnoremap <leader>h :History<CR>
```

Neovim and Modern Configuration

Neovim is a modernized Vim fork that's become the preferred choice for new users. It supports Lua for configuration (faster, more powerful than Vimscript).

```lua
-- ~/.config/nvim/init.lua (Neovim Lua config)
-- This is equivalent to vimrc but in Lua

-- Basic settings
vim.opt.number = true
vim.opt.relativenumber = true
vim.opt.expandtab = true
vim.opt.tabstop = 4
vim.opt.shiftwidth = 4
vim.opt.ignorecase = true
vim.opt.smartcase = true
vim.opt.hlsearch = true
vim.opt.incsearch = true
vim.opt.scrolloff = 8
vim.opt.clipboard = "unnamedplus"

-- Set leader key
vim.g.mapleader = " "

-- Keymaps
local map = vim.keymap.set
map("n", "<leader>w", ":w<CR>")
map("n", "<leader>q", ":q<CR>")
map("n", "<C-h>", "<C-w>h")
map("n", "<C-j>", "<C-w>j")
map("n", "<C-k>", "<C-w>k")
map("n", "<C-l>", "<C-w>l")
```

Hands-on Exercise 9: Setting Up Plugins

```
# 1. Add vim-plug to your vimrc (as shown above)
# 2. Add these plugins to start:
#    - tpope/vim-surround
#    - tpope/vim-commentary
#    - morhetz/gruvbox

# 3. Install plugins:
vim
:PlugInstall
# Wait for installation to complete

# 4. Set colorscheme:
:colorscheme gruvbox

# 5. Test vim-surround:
#    Type "hello" in a file
#    Move cursor to "hello"
#    Press ysiw" → should become "hello"
#    Press cs"' → should become 'hello'
#    Press ds' → should become hello

# 6. Test vim-commentary:
#    Open any code file
#    Press gcc → should comment out the line
#    Press gcc again → should uncomment
```

Section 3.3

Real-World Vim Workflows

🎯 Learning Objectives

- Apply Vim to real programming tasks efficiently
- Build a complete development workflow
- Use Vim for common developer tasks (Git, grep, refactoring)
- Understand when NOT to use Vim (and how to exit gracefully)

The Vim Development Workflow

```
A PROFESSIONAL VIM WORKFLOW
═══════════════════════════════════════════════════════════════

OPENING A PROJECT:
  cd my-project
  vim .          ← Opens netrw
  OR
  vim -p src/*.js tests/*.test.js  ← Open multiple files in tabs

NAVIGATING:
  <leader>f      ← FZF: fuzzy find any file
  <leader>g      ← Ripgrep: search across all files
  Ctrl+^         ← Toggle between last two files
  :ls → :b2      ← Switch to specific buffer

EDITING:
  ci"            ← Change inside quotes (most common)
  ysiw(          ← Wrap word in parens (vim-surround)
  gcc            ← Toggle line comment (vim-commentary)
  ggvG=          ← Select all, auto-indent
  :%s/old/new/gc ← Find and replace with confirmation

GIT INTEGRATION (vim-fugitive):
  :Git status    ← See status
  :Git add %     ← Stage current file
  :Git commit    ← Opens commit message editor
  :Git push      ← Push
  :Git diff      ← See diff
```

```
MULTIPLE FILES:
  :vsp utils.js  ← Open side by side
  Ctrl+w v       ← Vertical split current buffer
  Ctrl+w =       ← Equal size windows

PRODUCTIVITY PATTERNS:
  qa...q  @a     ← Record and replay macros for repetitive edits
  .              ← Repeat last change everywhere
  *n.n.n.        ← Find word, repeat changes on each
```

Common Real-World Tasks

Task 1: Find and Replace a Variable Name Across Project

```
# Option 1: Vim's built-in (current file)
:%s/\bOldName\b/NewName/gc   # \b = word boundary, c = confirm each

# Option 2: Using FZF + quickfix list
:Rg OldName         # Find all occurrences with ripgrep
# Select all results in FZF (Tab to select, Enter)
# Use :cfdo %s/OldName/NewName/g | w  to replace in all matched files

# Option 3: Using args
:args src/**/*.js    # Set all JS files as args
:argdo %s/\bOldName\b/NewName/ge | w   # Replace in all, save each
```

Task 2: Working with JSON

```
# Prettify JSON in current buffer:
:%!python3 -m json.tool

# Minify JSON:
:%!python3 -c "import sys,json;print(json.dumps(json.load(sys.stdin)))"

# Query JSON with jq (if installed):
:%!jq '.users[] | {name, email}'
```

Task 3: Aligning Code

```
# Before:
const a = 1;
const longVariable = 2;
const x = 3;

# Select all lines, then:
:'<,'>!column -t

# After (aligned):
const  a             =  1;
const  longVariable  =  2;
const  x             =  3;
```

Task 4: Working with Git from Vim

```
" With vim-fugitive installed:
:Git                  " Opens git status pane
:Git add %            " Stage current file
:Git commit           " Commit (opens editor for message)
:Git push             " Push to remote
:Git log              " View commit log
:Git blame            " Line-by-line blame view
:Gdiff                " Side-by-side diff with last commit
:Gread                " Revert current file to HEAD
```

Task 5: Refactoring Code with Macros

```
// BEFORE: Converting class methods to arrow functions

class Calculator {
    add(a, b) {
        return a + b;
    }
    subtract(a, b) {
        return a - b;
    }
    multiply(a, b) {
        return a * b;
    }
}
```

```
# MACRO to convert "    method(a, b) {" to "    method = (a, b) => {"

# Position cursor on "add" (line with the method)
qa                    # Start recording into 'a'
^                     # Go to first non-blank
e                     # Move to end of method name
a =                                           # Append " = " after method name
Esc
f(                    # Find the opening paren
i                     # Insert before (
Esc
f{                    # Find opening brace
s =>  {               # Replace { with " => {"
Esc
j^                    # Go to next line
q                     # Stop recording

# Apply to next 2 methods:
2@a
```

Tips, Tricks & Pro Insights

```
PRO-LEVEL VIM TIPS
═══════════════════════════════════════════════════════════════

TIP 1: THE FAST SEARCH-AND-EDIT LOOP
  *          ← Search for word under cursor
  ciw        ← Change inner word (make your edit)
  Esc
  n.         ← Go to next match, repeat the change
  n.n.n.     ← Repeat as needed

TIP 2: LAST INSERT REPEAT
  .          ← Repeats last INSERT operation too
  So: iHello<Esc> then . anywhere → inserts "Hello" again!

TIP 3: PASTE IN INSERT MODE
  While typing: Ctrl+r "    ← pastes from unnamed register
  While typing: Ctrl+r 0    ← pastes from yank register (last y)
  While typing: Ctrl+r %    ← pastes current filename!

TIP 4: LINE COMPLETION IN INSERT MODE
  Ctrl+x Ctrl+l    ← Complete current line from elsewhere in file
  Ctrl+x Ctrl+f    ← Complete filename
  Ctrl+x Ctrl+o    ← Omniscient completion (with plugins)
  Ctrl+n           ← Simple keyword completion (next match)
  Ctrl+p           ← Simple keyword completion (previous match)

TIP 5: JUMP TO DEFINITION / LAST CHANGE
  gd               ← Go to local definition of word under cursor
```

```
  gD               ← Go to global definition
  g;               ← Jump to last changed position
  g,               ← Jump to next change position (reverse of g;)

TIP 6: COMMAND HISTORY
  q:               ← Open command-line history as a buffer!
                     (Edit any past command and press Enter to run)
  q/               ← Open search history as a buffer!

TIP 7: SORT A SELECTION
  Visually select lines, then:
  :'<,'>sort        ← Sort alphabetically
  :'<,'>sort!       ← Sort in reverse
  :'<,'>sort n      ← Sort numerically

TIP 8: ZERO-EFFORT FILE COMPARISON
  vim -d file1.txt file2.txt    ← Diff mode: shows differences
  :diffsplit file2.txt           ← Split diff with another file
  ]c                             ← Jump to next diff
  [c                             ← Jump to previous diff
  do                             ← Diff obtain (copy from other)
  dp                             ← Diff put (copy to other)
```

Common Mistakes and How to Avoid Them

```
COMMON VIM MISTAKES
═══════════════════════════════════════════════════════════

MISTAKE 1: Using arrow keys
  ❌ Reaching for arrow keys constantly
  ✅ Force yourself to use hjkl for 1 week — then it's automatic

MISTAKE 2: Using Insert mode for navigation
  ❌ Pressing Esc, moving, pressing i again for every edit
  ✅ Normal mode is for navigation. Use ci", cw, etc. to change in place

MISTAKE 3: Not using counts
  ❌ jjjjjjjj (pressing j 8 times)
  ✅ 8j OR better: use relative line numbers and jump directly

MISTAKE 4: Not using text objects
  ❌ vh (visual select by characters) then d
  ✅ diw, daw, di{, di( — semantic deletion

MISTAKE 5: Not using the dot command
  ❌ Repeating the same edit manually 20 times
  ✅ Do it once, use . to repeat everywhere

MISTAKE 6: Huge, unfocused plugins
  ❌ Installing 50 plugins and understanding none of them
  ✅ Start with 3-5 plugins. Master each before adding more.

MISTAKE 7: Never reading :help
```

```
   ❌ Searching Google for every Vim question
   ✅ :help {topic} is comprehensive and always available
      :help usr_01  ← Start the built-in Vim tutorial

MISTAKE 8: Exiting Vim incorrectly
   ❌ Killing the terminal, Ctrl+Z, force-quitting
   ✅ :q (no changes), :q! (discard changes), :wq (save and quit)
```

The Vim Learning Curve — Realistic Expectations

```
YOUR VIM JOURNEY
═══════════════════════════════════════════════════════════

WEEK 1: Confusion → Survival
  ✅ You know how to enter/exit Insert mode
  ✅ You can save and quit
  ✅ hjkl movement (still awkward)
  ✅ Basic dd, yy, p
  ❌ You're slower than your old editor — THIS IS NORMAL

WEEK 2-3: Basics → Competence
  ✅ hjkl feels natural
  ✅ w, b, e, 0, $ feel useful
  ✅ You reach for d/c/y automatically
  ✅ Text objects start making sense
  ⚡ You start to feel the speed potential

MONTH 1-2: Competence → Fluency
  ✅ You rarely use the mouse
  ✅ / search is your primary navigation
  ✅ . repeat saves you constantly
  ✅ You have a working vimrc
  ⚡ You're as fast as your old editor (sometimes faster)

MONTH 3-6: Fluency → Flow
  ✅ You think in Vim commands before your fingers move
  ✅ Macros for repetitive tasks are natural
  ✅ Plugin ecosystem fits your workflow
```

```
   ⚡ Vim is noticeably faster for most editing tasks

YEAR 1+: Flow → Mastery
   ✅ Vim keybindings everywhere (IDE, browser, shell)
   ✅ You teach others and notice what you do instinctively
   ✅ You contribute vimrc snippets online
   ⚡ Editing at the speed of thought — it feels like magic

IMPORTANT: Every Vim expert was once helpless in Normal mode.
           The curve is steep but the plateau is very high.
```

Complete Vim Cheat Sheet

```
═══════════════════════════════════════════════════════════════
                    VIM COMPLETE CHEAT SHEET
═══════════════════════════════════════════════════════════════

MODES
  Normal    → Esc / Ctrl+c (from any mode)
  Insert    → i a I A o O s S cc C
  Visual    → v (char) V (line) Ctrl+v (block)
  Command   → : / ?

NAVIGATION
  h j k l        ← ↓ ↑ →
  w W b B e E    word/WORD forward/back/end
  0 ^ $          line start/non-blank/end
  gg G           file start/end
  nG :n          go to line n
  H M L          screen top/middle/bottom
  Ctrl+f/b       page down/up
  Ctrl+d/u       half page down/up
  f{c} F{c}      find char forward/backward
  t{c} T{c}      till char forward/backward
  ; ,            repeat f/F/t/T
  * #            search word under cursor fwd/back
  zz zt zb       center/top/bottom cursor on screen
  Ctrl+o Ctrl+i  jump back/forward
  m{c} '{c} `{c} set/jump-line/jump-exact mark

INSERT MODE ENTRY
```

```
  i I            insert before cursor / line start
  a A            append after cursor / line end
  o O            new line below / above
  s S            substitute char / line
  cc C           change line / to end

OPERATORS (combine with motion or text object)
  d  delete      y  yank    c  change
  >  indent      <  dedent  =  auto-indent
  gU uppercase   gu lowercase  g~ toggle case
  !  filter through command

TEXT OBJECTS (after operator: d y c > < = gU gu g~)
  iw aw   inner/a word
  iW aW   inner/a WORD
  is as   inner/a sentence
  ip ap   inner/a paragraph
  i" a"   inner/a double quotes
  i' a'   inner/a single quotes
  i( a(   inner/a parentheses  (also ib ab)
  i{ a{   inner/a curly braces (also iB aB)
  i[ a[   inner/a square brackets
  it at   inner/a HTML tag

COMMON COMMANDS
  x X          delete char under/before cursor
  dd D         delete line / to end of line
  yy Y         yank line / to end of line
  p P          paste after/before cursor
  u Ctrl+r     undo / redo
  . (dot)      repeat last change
  ~ g~ gU gu   toggle/toggle/upper/lower case
```

```
  J gJ         join line with next (with/without space)
  > < ==       indent right/left/auto
  ZZ ZQ        save+quit / quit no save

SEARCH & REPLACE
  /pattern     search forward
  ?pattern     search backward
  n N          next/previous match
  :noh         clear highlighting
  :%s/old/new/g  replace all in file
  :%s/old/new/gc replace with confirmation
  :g/pat/d     delete all matching lines
  :v/pat/d     delete all non-matching lines

REGISTERS
  "ayy  "ap    yank/paste using register 'a'
  "+yy  "+p    yank/paste system clipboard
  :reg         show all registers

MACROS
  qa...q       record macro into 'a'
  @a           play macro 'a'
  @@           replay last macro
  5@a          play 5 times

WINDOWS & BUFFERS
  :e file      open file
  :sp :vsp     horizontal/vertical split
  Ctrl+w hjkl  navigate windows
  Ctrl+w =     equal size windows
  Ctrl+^       toggle last two buffers
  :ls :b n     list buffers / switch to n
```

```
  :bd          close buffer

TABS
  :tabnew  gt gT  new tab / next / previous

FILES & SAVING
  :w           save
  :w file      save as
  :q :q!       quit / force quit
  :wq ZZ       save and quit
  :e!          revert to saved version

USEFUL EX COMMANDS
  :set nu/rnu  line numbers / relative
  :set ic/noic ignorecase / case sensitive
  :!command    run shell command
  :%!sort      sort file through sort command
  :r !cmd      insert command output at cursor
  :help topic  open help
  :help usr_01 built-in tutorial

═══════════════════════════════════════════════════════════════
```

Final Assignment: The Vim Workflow Challenge

```
ASSIGNMENT: BUILD AND EDIT A PROJECT ENTIRELY IN VIM
═══════════════════════════════════════════════════════════════

Create a "Student Grade Calculator" with these requirements,
using ONLY Vim commands (no mouse, no arrow keys):

PART 1: FILE CREATION
  - Create project directory and files using :! commands
  - Use Vim's netrw to navigate

PART 2: CODING
  - Write a JavaScript/Python grade calculator
  - Use text objects for editing (ci", ci{, diw etc.)
  - Use macros to create repetitive structure
  - Use visual block for aligned declarations

PART 3: REFACTORING
  - Rename a variable throughout the file using * and n.
  - Use :%s to change a function signature everywhere
  - Use :g to find and list all TODO comments

PART 4: FILE MANAGEMENT
  - Open the spec file in a vertical split (:vsp)
  - Use buffers to switch between files (Ctrl+^)
  - Edit both files simultaneously

GRADE YOURSELF:
  ✅ Never used arrow keys         → Expert Navigation
```

```
✅ Used 5+ text objects          → Expert Editing
✅ Used at least 1 macro         → Automation
✅ Used search + . to refactor   → Efficient Editing
✅ Used splits for multi-file    → Professional Workflow
```

Additional Resources

Resource	URL / Command	Best For
Built-in Tutorial	`vimtutor` in terminal	First 30 minutes
Vim Help	`:help {topic}`	Any time you have questions
Learn Vim the Hard Way	learnvimscriptthehardway.stevelosh.com	Vimscript and config
Vim Adventures	vim-adventures.com	Gamified navigation practice
Open Vim	openvim.com	Interactive in-browser tutorial
Practical Vim	Book by Drew Neil	Deep dive, patterns
Neovim Docs	neovim.io/doc	Modern Vim reference
r/vim	reddit.com/r/vim	Community, tips, configs
Awesome Neovim	github.com/rockerBOO/awesome-neovim	Plugin discovery
Neovim from Scratch	YouTube: TJ DeVries	Modern Neovim setup

Program Wrap-Up

What You've Accomplished in 3 Days

```
DAY 1 MASTERED:                 DAY 2 MASTERED:                 DAY 3 MASTERED:
═══════════════                 ═══════════════                 ═══════════════
✅ Vim philosophy               ✅ Operators + motions          ✅ Buffers/windows/tabs
✅ Modal editing                ✅ Text objects                 ✅ Plugins (vim-plug)
✅ Normal mode survival         ✅ The dot command              ✅ Advanced editing
✅ hjkl navigation              ✅ Search & replace             ✅ Macros (expert)
✅ Basic insert/delete          ✅ Registers                    ✅ Real-world workflow
✅ Yank and paste               ✅ Basic macros                 ✅ vimrc mastery
✅ Undo/redo                    ✅ vimrc configuration          ✅ Visual block
✅ File operations              ✅ Colorschemes                 ✅ Git integration
```

The Vim Mindset

```
╔═══════════════════════════════════════════════════════════════╗
║                                                               ║
║   HJKL BEFORE ARROW KEYS                                      ║
║   → One week of pain, lifetime of gain                        ║
║                                                               ║
║   NORMAL MODE IS HOME                                         ║
║   → Don't live in Insert mode. Be a visitor, not a resident.   ║
║                                                               ║
║   THE DOT COMMAND IS YOUR BEST FRIEND                         ║
║   → Make it work once. Make it work everywhere.               ║
║                                                               ║
║   OPERATORS + MOTIONS = INFINITE POWER                         ║
║   → Learn the grammar, not the dictionary                      ║
║                                                               ║
║   READ THE SOURCE (i.e., :help)                                ║
║   → Vim's help is the best editor docs ever written            ║
║                                                               ║
╚═══════════════════════════════════════════════════════════════╝
```

End of Mastering the Vim Editor — 3-Day Intensive Training Guide

Written for beginners. Designed for mastery. Built for developers who demand speed.